Wormhole Addresses: Scalable Post-Quantum Privacy-Preserving Transactions via Recursive Zero-Knowledge Proofs

Christopher Smith Ethan Cemer

March 10, 2026

Abstract

The transition to post-quantum cryptography presents a significant scalability challenge for blockchain systems. Post-quantum signature schemes such as ML-DSA (Dilithium) require public keys and signatures that are 50–100× larger than their elliptic curve counterparts, severely limiting transaction throughput. We introduce *wormhole addresses*, a novel construction that combines provably unspendable addresses with recursive zero-knowledge proofs to enable scalable, privacy-preserving transactions without revealing large post-quantum signatures on-chain. Our protocol leverages Plonky2, a FRI-based proof system requiring no trusted setup, to achieve post-quantum security while enabling recursive proof aggregation. We demonstrate that wormhole transactions can achieve over 60× throughput improvement compared to naive post-quantum signature verification, while simultaneously providing unlinkability guarantees similar to mixing protocols. We provide formal security definitions and proofs for double-spend resistance, unspendability, and transaction privacy. The system is implemented and deployed on the Quantus Network, a production blockchain.

1 Introduction

The emergence of practical quantum computers poses an existential threat to the cryptographic foundations of modern blockchain systems. Elliptic curve cryptography, which underpins the security of Bitcoin, Ethereum, and virtually all major cryptocurrencies, is vulnerable to Shor's algorithm [9]. In response, the cryptographic community has developed post-quantum signature schemes, with NIST standardizing ML-DSA (formerly CRYSTALS-Dilithium) [7] as the primary replacement for classical signatures.

However, the transition to post-quantum cryptography introduces a severe scalability problem. While Ed25519 signatures require only 64 bytes with 32-byte public keys, ML-DSA-87 (the highest security level) requires 4,627-byte signatures with 2,592-byte public keys—a combined overhead increase of nearly 75×. For a blockchain with 5MB blocks, this reduces maximum throughput from tens of thousands of transactions per block to fewer than 700.

1.1 Our Contribution

We propose *wormhole addresses*, a construction that fundamentally reimagines how transaction authorization works in post-quantum blockchains. Rather than verifying signatures on-chain, users deposit funds to *provably unspendable addresses* and later withdraw using zero-knowledge proofs that demonstrate knowledge of a secret value linking them to the deposit. The key insight is that proving ownership via a ZK proof requires only $O(1)$ on-chain verification regardless of the underlying signature scheme's size.

Our contributions are as follows:

1. **Wormhole Address Construction:** We introduce a double-hashing construction $\mathcal{H}(\mathcal{H}(\mathsf{salt}\|s))$ that creates addresses provably unspendable under any signature scheme with public keys larger than the hash preimage size.

2. **Privacy-Preserving Withdrawals:** We design a zero-knowledge circuit that proves fund ownership while hiding which specific deposit is being spent, providing unlinkability guarantees.

3. **Recursive Proof Aggregation:** We develop a multi-layer aggregation scheme that batches thousands of withdrawals into a single on-chain proof verification, with careful handling of dummy proofs to preserve privacy.

4. **Formal Security Analysis:** We provide rigorous security definitions and proofs for double-spend resistance, unspendability, and privacy against balance-linking attacks.

5. **Production Implementation:** We implement and deploy the system on the Quantus Network using Plonky2 [11], achieving post-quantum security without trusted setup.

1.2 Technical Overview

The wormhole protocol operates in three phases. First, a user generates a random secret s and derives a *wormhole address* $\alpha = \mathcal{H}(\mathcal{H}(\text{“}\mathsf{wormhole}''\|s))$. This address appears as a normal account but is provably unspendable because no valid signature scheme can produce a public key that hashes to α's preimage. Second, the user (or anyone) deposits funds to α via standard transactions. Third, to withdraw, the user generates a zero-knowledge proof demonstrating: (1) knowledge of secret s that derives α, (2) existence of a deposit to α in the blockchain state, and (3) a fresh nullifier preventing double-spending.

The zero-knowledge proof is constructed using Plonky2, a recursive SNARK based on FRI (Fast Reed-Solomon Interactive Oracle Proofs of Proximity) [1]. Unlike pairing-based SNARKs, FRI-based systems require no trusted setup ceremony and achieve post-quantum security, as their soundness relies only on the collision resistance of hash functions.

To maximize throughput, we introduce a recursive aggregation scheme where multiple withdrawal proofs are combined into a single proof. This aggregation is privacy-sensitive: to prevent an observer from determining which proofs in a batch are real versus padding (dummy proofs), we employ careful sentinel value design and proof shuffling at the privacy-critical layer.

1.3 Paper Organization

Section 2 introduces notation and cryptographic preliminaries. Section 3 defines our security model and adversarial assumptions. Section 4 presents the wormhole protocol in detail. Section 5 provides security proofs. Section 6 describes our privacy score heuristic. Section 7 discusses implementation. Section 8 surveys related work, and Section 9 concludes.

2 Preliminaries

2.1 Notation

We denote by $\{0,1\}^n$ the set of n-bit strings and by $\{0,1\}^*$ the set of all finite binary strings. For a finite set S, we write $x \leftarrow S$ to denote sampling x uniformly at random from S. We use λ to

denote the security parameter. A function $\mathsf{negl}(\lambda)$ is *negligible* if for every polynomial $p(\cdot)$, there exists N such that for all $\lambda > N$, $\mathsf{negl}(\lambda) < 1/p(\lambda)$.

We work over the Goldilocks field $\mathbb{F}_p$ where $p = 2^{64} - 2^{32} + 1$. This 64-bit prime enables efficient arithmetic on modern processors while providing adequate security margins. Field elements are denoted by lowercase letters (a, b, x), and vectors of field elements by bold lowercase $(\mathbf{a}, \mathbf{x})$.

2.2 Hash Functions

Our construction requires a cryptographic hash function $\mathcal{H} : \{0,1\}^* \to \mathbb{F}_p^4$ that is:

- **Collision Resistant:** For all PPT adversaries $\mathcal{A}$, $\Pr[(x, x') \leftarrow \mathcal{A}(1^\lambda) : x \neq x' \wedge \mathcal{H}(x) = \mathcal{H}(x')] \leq \mathsf{negl}(\lambda)$.
- **Preimage Resistant:** For all PPT adversaries $\mathcal{A}$ and random $y \leftarrow \mathbb{F}_p^4$, $\Pr[x \leftarrow \mathcal{A}(1^\lambda, y) : \mathcal{H}(x) = y] \leq \mathsf{negl}(\lambda)$.

We instantiate $\mathcal{H}$ with Poseidon2 [5], an algebraic hash function designed for efficient evaluation inside arithmetic circuits. Poseidon2 operates natively over prime fields, avoiding the overhead of bit decomposition required by traditional hash functions like SHA-256. The hash outputs a 256-bit digest represented as four 64-bit field elements.

2.3 Zero-Knowledge Proof Systems

Definition 1 (Zero-Knowledge Argument)**.** *A zero-knowledge argument system for a relation $\mathcal{R}$ consists of algorithms* $(\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$ *such that:*

- $\mathsf{Setup}(1^\lambda, \mathcal{R}) \to \mathsf{pp}$*: Generates public parameters.*
- $\mathsf{Prove}(\mathsf{pp}, x, w) \to \pi$*: Given instance x and witness w with $(x, w) \in \mathcal{R}$, produces proof π.*
- $\mathsf{Verify}(\mathsf{pp}, x, \pi) \to \{0,1\}$*: Verifies the proof.*

The system must satisfy:

- **Completeness:** For all $(x, w) \in \mathcal{R}$, $\Pr[\mathsf{Verify}(\mathsf{pp}, x, \mathsf{Prove}(\mathsf{pp}, x, w)) = 1] = 1$.
- **Knowledge Soundness:** For any PPT prover $\mathcal{P}^*$ producing accepting proofs, there exists an extractor $\mathcal{E}$ that outputs a valid witness with overwhelming probability.
- **Zero-Knowledge:** There exists a simulator $\mathcal{S}$ that, given only x, produces proofs indistinguishable from real proofs.

2.4 Plonky2 and FRI

We build our protocol on Plonky2 [11], a recursive SNARK combining PLONK arithmetization with FRI-based polynomial commitments [1]. Key properties include:

- **Transparent Setup:** No trusted setup ceremony is required; public parameters are deterministically derived.
- **Post-Quantum Security:** Security relies on the hardness of finding low-degree polynomials close to random functions, not on discrete logarithm or factoring assumptions.

- **Efficient Recursion:** Proofs can verify other proofs, enabling aggregation with logarithmic overhead.
- **Configurable Security:** With extension degree $D = 2$ over $\mathbb{F}_p$, Plonky2 achieves approximately 100 bits of security.

FRI (Fast Reed-Solomon Interactive Oracle Proofs of Proximity) is an interactive oracle proof that a committed function is close to a low-degree polynomial. The prover commits to evaluations of a polynomial and recursively folds the polynomial until the verifier can directly check a constant polynomial. Security follows from the proximity gaps of Reed-Solomon codes.

2.5 Merkle-Patricia Tries

Blockchain state is stored in a Merkle-Patricia Trie (MPT), a authenticated data structure combining a radix trie with Merkle hashing. Each node's hash commits to its children, creating a root hash that authenticates the entire state.

An *inclusion proof* for key-value pair (k, v) consists of the path from the root to the leaf, along with sibling hashes at each level. Verification recomputes the root hash and compares against the trusted state root. Our circuit verifies MPT proofs using Poseidon2 for node hashing, enabling efficient ZK verification of on-chain state.

3 Security Model

We define the security properties required of a wormhole transaction system. Our model considers an adversary $\mathcal{A}$ that controls the network and can create arbitrary transactions, but cannot break the underlying cryptographic primitives.

3.1 System Model

The system consists of:

- A set of *accounts* $\mathcal{A} \subseteq \mathbb{F}_p^4$, each identified by a 256-bit address.
- A *state* $\sigma : \mathcal{A} \to \mathbb{Z}_{\geq 0}$ mapping accounts to balances.
- A *nullifier set* $\mathcal{N} \subseteq \mathbb{F}_p^4$ tracking spent deposits.
- A *blockchain* $\mathcal{B} = (B_0, B_1, \ldots, B_n)$ where each block B_i contains a state root rt_i and extrinsics root.

Accounts are partitioned into *spendable accounts* (controlled by a signature key pair) and *wormhole accounts* (provably unspendable, controlled by knowledge of a secret).

3.2 Security Definitions

Definition 2 (Unspendability)**.** *A wormhole address α is* unspendable *if no PPT adversary $\mathcal{A}$ can produce a valid signature σ under any signature scheme Σ such that $\mathsf{Verify}_\Sigma(\mathsf{pk}, m, \sigma) = 1$ and $\mathsf{Addr}(\mathsf{pk}) = \alpha$, except with negligible probability.*

Intuitively, unspendability ensures that funds deposited to a wormhole address cannot be withdrawn via the standard transaction mechanism—only via a valid zero-knowledge proof.

Definition 3 (Double-Spend Resistance)**.** *A wormhole system is* double-spend resistant *if for any PPT adversary $\mathcal{A}$ that deposits amount v to a wormhole address and produces polynomially many withdrawal proofs $\{\pi_i\}$, the total withdrawn amount does not exceed v except with negligible probability.*

Definition 4 (Withdrawal Soundness)**.** *A wormhole system has* withdrawal soundness *if for any PPT adversary $\mathcal{A}$ producing an accepting withdrawal proof π with public inputs $(n, \alpha_{\mathsf{exit}}, v, \mathsf{rt})$, there exists a deposit of at least v to some wormhole address in the state authenticated by rt, except with negligible probability.*

Definition 5 (Transaction Privacy)**.** *A wormhole system provides* transaction privacy *if for any PPT adversary $\mathcal{A}$ observing the blockchain, given a withdrawal of amount v, the adversary cannot determine which deposit funded the withdrawal with probability significantly better than random guessing among all deposits that could sum to v.*

3.3 Adversarial Model

We consider a computationally bounded adversary with the following capabilities:

- **Network Control:** $\mathcal{A}$ can observe all transactions and reorder them within blocks (but cannot censor indefinitely).
- **Deposit Creation:** $\mathcal{A}$ can create deposits to any address, including wormhole addresses it does not control.
- **Proof Observation:** $\mathcal{A}$ observes all withdrawal proofs and their public inputs (nullifiers, exit accounts, amounts).
- **Timing Analysis:** $\mathcal{A}$ can correlate deposit and withdrawal times.

The adversary *cannot*:

- Break collision or preimage resistance of $\mathcal{H}$.
- Break knowledge soundness of the ZK proof system.
- Learn secrets of honest users (no side-channel attacks).

3.4 Trust Assumptions

Our system requires:

1. **Hash Function Security:** Poseidon2 provides $\geq$ 128-bit collision resistance and preimage resistance.
2. **Proof System Soundness:** Plonky2/FRI achieves $\geq$ 100-bit soundness.
3. **Blockchain Liveness:** The underlying blockchain eventually includes valid transactions.
4. **State Availability:** Users can obtain Merkle proofs for their deposits.

Notably, we require *no trusted setup*—all parameters are publicly derivable.

4 The Wormhole Protocol

This section presents the wormhole protocol in detail, including address derivation, the withdrawal circuit, and recursive proof aggregation.

4.1 Wormhole Address Derivation

A wormhole address is derived from a user's secret $s \in \{0,1\}^{256}$ via double hashing:

$$\alpha = \mathcal{H}(\mathcal{H}(\text{“}\mathsf{wormhole}''\|s)) \tag{1}$$

The double-hashing construction is crucial for security. Given address α, an attacker attempting to spend funds via a signature must find a public key pk such that $\mathsf{Addr}(\mathsf{pk}) = \alpha$. Since addresses are derived by hashing public keys, this requires finding the preimage of the preimage: $\mathcal{H}^{-1}(\mathcal{H}^{-1}(\alpha))$.

Lemma 1. *If $\mathcal{H}$ is preimage resistant and the minimum public key size of signature scheme Σ exceeds 256 bits, then wormhole addresses are unspendable under Σ.*

Proof sketch. A valid public key pk for ML-DSA-87 is 2,592 bytes. For $\mathsf{Addr}(\mathsf{pk}) = \alpha$, we need $\mathcal{H}(\mathsf{pk}) = \mathcal{H}^{-1}(\alpha)$. The preimage $\mathcal{H}^{-1}(\alpha)$ is 256 bits (the hash output size). Since $|\mathsf{pk}| = 2592 \times 8 = 20736$ bits $\gg 256$ bits, no valid public key can equal a 256-bit preimage. Thus finding a spending key requires either (1) finding a hash collision to map a large key to a small preimage, or (2) breaking preimage resistance twice. □

4.2 Nullifier Derivation

To prevent double-spending, each withdrawal publishes a *nullifier* derived deterministically from the secret and a transfer counter:

$$n = \mathcal{H}(\mathcal{H}(\text{“} \sim \mathsf{nullif} \sim''\|s\|c)) \tag{2}$$

where c is a monotonically increasing counter per wormhole address. The on-chain nullifier set $\mathcal{N}$ tracks all published nullifiers; a withdrawal is rejected if its nullifier already exists in $\mathcal{N}$.

The counter c is stored on-chain in a `TransferProof` record associated with each deposit, enabling multiple withdrawals from addresses that receive multiple deposits.

4.3 The Wormhole Circuit

The zero-knowledge circuit proves the following relation:

$$\mathcal{R}_{\mathsf{wormhole}} = \left\{ \begin{array}{l} (\mathbf{x}, \mathbf{w}) : \\ \quad \mathbf{x} = (a, v_1, v_2, f, n, \alpha_1, \alpha_2, h_B, b) \\ \quad \mathbf{w} = (s, \pi_{\mathsf{MPT}}, c, \alpha_{\mathsf{fund}}, \alpha_{\mathsf{wh}}, \mathsf{rt}, \mathsf{hdr}, v_{\mathsf{in}}) \\ \quad \wedge \text{ constraints (C1)–(C6) hold} \end{array} \right\} \tag{3}$$

Public Inputs x:

- $a \in \mathbb{F}_p$: Asset identifier (0 for native token)
- $v_1, v_2 \in \mathbb{Z}_{2^{32}}$: Output amounts (quantized to 0.01 precision)
- $f \in \mathbb{Z}_{10000}$: Volume fee in basis points

- $n \in \mathbb{F}_p^4$: Nullifier (256 bits)
- $\alpha_1, \alpha_2 \in \mathbb{F}_p^4$: Exit account addresses
- $h_B \in \mathbb{F}_p^4$: Block hash commitment
- $b \in \mathbb{Z}_{2^{32}}$: Block number

Private Inputs w:

- $s \in \{0,1\}^{256}$: User's secret
- π_{MPT}: Merkle-Patricia trie inclusion proof
- $c \in \mathbb{Z}$: Transfer counter
- $\alpha_{\mathsf{fund}}, \alpha_{\mathsf{wh}} \in \mathbb{F}_p^4$: Funding and wormhole account addresses
- $\mathsf{rt} \in \mathbb{F}_p^4$: State root
- hdr: Block header fields (parent hash, extrinsics root, digest)
- $v_{\mathsf{in}} \in \mathbb{Z}$: Input amount (before fees)

Circuit Constraints:

(C1) Nullifier Derivation:

$$n = \mathcal{H}(\mathcal{H}(\text{“}\sim \mathsf{nullif} \sim\text{''} \| s \| c)) \tag{4}$$

(C2) Wormhole Account Derivation:

$$\alpha_{\mathsf{wh}} = \mathcal{H}(\mathcal{H}(\text{“}\mathsf{wormhole}\text{''} \| s)) \tag{5}$$

(C3) Storage Proof Verification: The MPT proof π_{MPT} is valid for the `TransferProof` record at $(\alpha_{\mathsf{fund}}, \alpha_{\mathsf{wh}}, c)$ under state root rt.

(C4) Block Header Commitment:

$$h_B = \mathcal{H}(\mathsf{parent_hash} \| b \| \mathsf{rt} \| \mathsf{extrinsics_root} \| \mathsf{digest}) \tag{6}$$

(C5) Fee Constraint:

$$(v_1 + v_2) \cdot 10000 \leq v_{\mathsf{in}} \cdot (10000 - f) \tag{7}$$

(C6) Range Checks: $v_1, v_2, v_{\mathsf{in}} \in [0, 2^{32})$ and $b \in [0, 2^{32})$.

Dummy Proof Handling: For batch padding, dummy proofs use sentinel values: $h_B = \mathbf{0}$ *and* $v_1 = v_2 = 0$. The circuit skips constraints (C3)–(C4) for dummy proofs while still enforcing the remaining structure. Dummy nullifiers are set to random values to prevent on-chain collision.

4.4 Transaction Flow

The protocol operates in three phases:

Algorithm 1 Wormhole Deposit

procedure DEPOSIT(sender, v, s)
  $\alpha_{\mathsf{wh}} \leftarrow \mathcal{H}(\mathcal{H}(\text{“}\mathsf{wormhole}''\|s))$
  Submit transaction: $\mathsf{Transfer}(\mathsf{sender}, \alpha_{\mathsf{wh}}, v)$
  Chain creates `TransferProof` record with counter c
end procedure

Algorithm 2 Wormhole Withdrawal

procedure WITHDRAW($s, c, v_1, v_2, \alpha_1, \alpha_2$)
  Fetch current block header $(h_B, b, \mathsf{rt}, \ldots)$
  Fetch MPT proof π_{MPT} for `TransferProof`
  $n \leftarrow \mathcal{H}(\mathcal{H}(\text{“}\sim \mathsf{nullif} \sim''\|s\|c))$
  $\pi \leftarrow \mathsf{Prove}(\mathsf{pp}, \mathbf{x}, \mathbf{w})$
  Submit $(\pi, \mathbf{x})$ for on-chain verification
end procedure

Algorithm 3 On-Chain Verification

procedure VERIFYWITHDRAWAL($\pi, \mathbf{x}$)
  Parse $\mathbf{x} = (a, v_1, v_2, f, n, \alpha_1, \alpha_2, h_B, b)$
Require: h_B matches block b in chain history
Require: f matches current fee configuration
Require: $n \notin \mathcal{N}$ (nullifier not spent)
Require: $\mathsf{Verify}(\mathsf{pp}, \mathbf{x}, \pi) = 1$
  $\mathcal{N} \leftarrow \mathcal{N} \cup \{n\}$
  Mint v_1 to α_1 and v_2 to α_2
end procedure

4.5 Recursive Proof Aggregation

To maximize throughput, we aggregate multiple withdrawal proofs into a single on-chain verification. The aggregation is structured as a tree with privacy-sensitive and privacy-safe layers.

4.5.1 Layer 0: Client-Local Aggregation

Layer 0 aggregates N leaf proofs from a single client. This layer is *privacy-sensitive* because it handles dummy proof positioning.

Algorithm 4 Layer-0 Aggregation

```
procedure AggregateLayer0({π_i}_{i=1}^N)
    Shuffle proofs, preserving first real proof position
    for each duplicate exit account pair do
        Sum amounts to single slot, zero the duplicate
    end for
    Verify all real proofs share same (h_B, a, f)
    π_agg ← recursive verification of {π_i}
    return (π_agg, x_agg)
end procedure
```

The Layer-0 output contains:

- Number of exit slots ($2N$ for two outputs per leaf)
- Asset ID, fee rate, block hash, block number
- Exit slots: $[(v_i, \alpha_i)]$ for each output
- Nullifiers from all N proofs

4.5.2 Layer 1: Delegated Aggregation

Layer 1 combines multiple Layer-0 proofs from different clients. This layer is *privacy-safe*—it cannot observe which proofs within a Layer-0 batch are dummies.

Algorithm 5 Layer-1 Aggregation

```
procedure AggregateLayer1({π_j^(0)}_{j=1}^M, α_agg)
    Verify all Layer-0 proofs share same (h_B, a, f)
    Concatenate exit slots from all children
    Concatenate nullifiers from all children
    π^(1) ← recursive verification of {π_j^(0)}
    Prepend aggregator address α_agg for fee collection
    return (π^(1), x^(1))
end procedure
```

4.5.3 Aggregation Tree Properties

The multi-layer structure achieves several goals:

1. **Scalability:** $O(\log k)$ on-chain verification for k withdrawals.
2. **Privacy Preservation:** Dummy positions hidden at Layer 0; higher layers see only opaque aggregates.
3. **Economic Incentives:** Aggregator addresses enable fee collection for batch operators.
4. **Parallelization:** Independent subtrees can be aggregated concurrently.

4.6 Two-Output Model

Each withdrawal produces two outputs (v_1, α_1) and (v_2, α_2), similar to Bitcoin's UTXO model. This enables:

- **Change Outputs:** Withdraw partial amounts while preserving remaining balance.
- **Split Payments:** Pay multiple recipients in a single proof.
- **Privacy Enhancement:** Splitting amounts increases ambiguity in balance-linking attacks.

Amounts are quantized to 0.01 units with a maximum of $2^{32} - 1$ units, supporting up to 21 million tokens with cent precision.

5 Security Analysis

We prove that the wormhole protocol satisfies the security properties defined in Section 3.

5.1 Double-Spend Resistance

Theorem 2 (Double-Spend Resistance)**.** *The wormhole protocol is double-spend resistant under the collision resistance of $\mathcal{H}$ and knowledge soundness of the ZK proof system.*

Proof. Suppose adversary $\mathcal{A}$ deposits amount v and produces two accepting proofs π_1, π_2 with nullifiers n_1, n_2 such that both pass on-chain verification.

Case 1: $n_1 = n_2$. The second proof is rejected because $n_1 \in \mathcal{N}$ after the first withdrawal.

Case 2: $n_1 \neq n_2$ for the same deposit. By knowledge soundness, there exist witnesses (s_1, c_1) and (s_2, c_2) such that:

$$n_1 = \mathcal{H}(\mathcal{H}(\mathsf{salt} \| s_1 \| c_1)) \tag{8}$$

$$n_2 = \mathcal{H}(\mathcal{H}(\mathsf{salt} \| s_2 \| c_2)) \tag{9}$$

For both proofs to reference the same deposit's `TransferProof` record (which contains a fixed counter c), we need $c_1 = c_2 = c$. If $s_1 = s_2 = s$ (same secret), then $n_1 = n_2$, contradicting our assumption.

If $s_1 \neq s_2$, then $\mathcal{A}$ found two secrets mapping to the same wormhole address:

$$\mathcal{H}(\mathcal{H}(\text{“}\mathsf{wormhole}''\| s_1)) = \mathcal{H}(\mathcal{H}(\text{“}\mathsf{wormhole}''\| s_2))$$

This constitutes a collision in $\mathcal{H}$, contradicting collision resistance.

Case 3: Different deposits. Each deposit has a unique `TransferProof` with its own counter. Withdrawing from multiple deposits is permitted (not double-spending).

Therefore, $\mathcal{A}$ cannot double-spend a single deposit. □

5.2 Unspendability

Theorem 3 (Unspendability)**.** *Wormhole addresses are unspendable under any signature scheme with public keys larger than the hash output size, assuming $\mathcal{H}$ is preimage resistant.*

Proof. Let $\alpha = \mathcal{H}(\mathcal{H}(\text{“}\mathsf{wormhole}''\|s))$ be a wormhole address. To spend via signature scheme Σ, adversary $\mathcal{A}$ must find key pair $(\mathsf{sk}, \mathsf{pk})$ such that $\mathsf{Addr}(\mathsf{pk}) = \alpha$.

Standard address derivation is $\mathsf{Addr}(\mathsf{pk}) = \mathcal{H}(\mathsf{pk})$. Thus $\mathcal{A}$ needs:

$$\mathcal{H}(\mathsf{pk}) = \alpha = \mathcal{H}(\mathcal{H}(\text{“}\mathsf{wormhole}''\|s))$$

This requires either:

1. Finding pk such that $\mathsf{pk} = \mathcal{H}(\text{“}\mathsf{wormhole}''\|s)$ (a 256-bit value). But valid public keys for ML-DSA are 2,592 bytes $= 20,736$ bits. A 256-bit string cannot be a valid ML-DSA public key.

2. Finding a collision: distinct $\mathsf{pk} \neq \mathcal{H}(\text{“}\mathsf{wormhole}''\|s)$ with $\mathcal{H}(\mathsf{pk}) = \mathcal{H}(\mathcal{H}(\text{“}\mathsf{wormhole}''\|s))$. This breaks collision resistance.

Both cases contradict our assumptions, so wormhole addresses are unspendable. □

5.3 Withdrawal Soundness

Theorem 4 (Withdrawal Soundness)**.** *If the ZK proof system is knowledge sound and $\mathcal{H}$ is collision resistant, then every accepting withdrawal proof corresponds to a valid deposit.*

Proof. Let π be an accepting proof with public inputs including block hash h_B and amounts v_1, v_2. By knowledge soundness, there exists a witness including:

- Secret s deriving the wormhole address α_{wh}
- Valid MPT proof π_{MPT} for a `TransferProof` record
- Input amount v_{in} satisfying the fee constraint

The circuit verifies:

1. π_{MPT} authenticates against state root rt
2. Block header hashes to h_B using rt
3. On-chain verification checks h_B matches a real block

Therefore, the `TransferProof` record exists in the authenticated blockchain state. The record is created only when a deposit transaction executes, so a valid deposit must exist.

The fee constraint ensures $v_1 + v_2 \leq v_{\mathsf{in}}$ (accounting for fees), so the withdrawn amount does not exceed the deposited amount. □

5.4 Privacy Analysis

We analyze privacy against balance-linking attacks, where an adversary attempts to correlate withdrawal amounts with specific deposits.

Definition 6 (Balance-Linking Advantage)**.** *The balance-linking advantage of adversary $\mathcal{A}$ is:*

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{link}} = \left| \Pr[\mathcal{A} \textit{ correctly links withdrawal to deposit}] - \frac{1}{|\mathsf{Candidates}|} \right|$$

where $\mathsf{Candidates}$ *is the set of deposits that could have funded the withdrawal.*

Theorem 5 (Privacy Under Aggregation)**.** *For a batch of k withdrawals with d dummy proofs, the balance-linking advantage is bounded by:*

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{link}} \leq \frac{1}{\binom{|\mathcal{D}|}{k-d}}$$

where $\mathcal{D}$ is the set of deposits with compatible amounts.

Proof sketch. The adversary observes: (1) total batch output amounts, (2) nullifiers (unlinkable to deposits), (3) exit accounts. The adversary does not observe which proofs are dummies or the input amounts.

For a batch output of total value V, the adversary must determine which subset of deposits sums to V (accounting for fees). This is a variant of the subset-sum problem. With $|\mathcal{D}|$ candidate deposits and $k - d$ real proofs, there are $\binom{|\mathcal{D}|}{k-d}$ possible subsets.

Additional privacy comes from:

- **Amount quantization:** Reduces distinguishing information.
- **Two-output splitting:** Each proof can split amounts arbitrarily.
- **Exit deduplication:** Duplicate exit accounts are summed, hiding individual contributions.
- **Dummy shuffling:** Real proof positions are randomized within the batch.

□

5.5 Dummy Proof Indistinguishability

Lemma 6 (Dummy Indistinguishability)**.** *An observer cannot distinguish real proofs from dummy proofs in a Layer-0 aggregate, assuming the ZK proof system is zero-knowledge.*

Proof. Dummy proofs use sentinel values: $h_B = \mathbf{0}$ and $v_1 = v_2 = 0$. After aggregation:

1. Exit slots with zero amounts are indistinguishable from real zero-change outputs.
2. Dummy nullifiers are random, indistinguishable from real nullifiers.
3. Proof shuffling randomizes positions.
4. Duplicate exit accounts are zeroed, matching dummy format.

The aggregated public inputs do not reveal which of the N constituent proofs were dummies. By zero-knowledge, the proof itself reveals nothing beyond the public inputs. □

5.6 Attack Mitigations

Timing Correlation: Users should introduce random delays between deposits and withdrawals, and withdraw to fresh addresses.

Amount Fingerprinting: The two-output model and quantization reduce unique amount patterns. Users can sacrifice precision for privacy by withdrawing round amounts.

Graph Analysis: Exit account reuse is discouraged. The system supports HD-wallet-style address derivation for fresh exit addresses per withdrawal.

6 Privacy Score Heuristic

While Section 5 provides asymptotic privacy guarantees, users benefit from a concrete metric estimating their anonymity. We present a heuristic *privacy score* that quantifies the difficulty of balance-linking attacks for a given withdrawal.

6.1 Anonymity Set Estimation

The privacy score estimates $\log_2$ of the anonymity set size—the number of deposit subsets that could plausibly have funded the observed withdrawal.

Definition 7 (Privacy Score)**.** *For a withdrawal of amount V from a deposit pool $\mathcal{D}$, the privacy score is:*

$$\mathsf{Score}(V, \mathcal{D}) = \log_2 \left| \left\{ S \subseteq \mathcal{D} : \sum_{d \in S} d \cdot (1 - f) \approx V \right\} \right|$$

where f is the fee rate and $\approx$ denotes equality within quantization tolerance.

Directly computing this requires enumerating exponentially many subsets. We use a Central Limit Theorem (CLT) approximation for efficiency.

6.2 CLT-Based Approximation

We partition deposits into buckets by amount range. For bucket B containing D deposits with mean μ and standard deviation σ, the sum of k randomly chosen deposits is approximately normal:

$$\sum_{i=1}^{k} d_i \sim \mathcal{N}(k\mu, \sigma\sqrt{k})$$

The probability that a k-subset sums to target range $[V_{\min}, V_{\max}]$ is:

$$P_k = \Phi\left(\frac{V_{\max} - k\mu}{\sigma\sqrt{k}}\right) - \Phi\left(\frac{V_{\min} - k\mu}{\sigma\sqrt{k}}\right)$$

where Φ is the standard normal CDF.

The expected number of valid subsets is:

$$\mathsf{Count} = \sum_{k=k_{\min}}^{k_{\max}} \binom{D}{k} \cdot P_k$$

To avoid numerical overflow, we compute in log-space:

$$\mathsf{Score} = \log_2 \left(\sum_k \exp_2 \left(\log_2 \binom{D}{k} + \log_2 P_k \right) \right)$$

6.3 Score Interpretation

Score (bits)	Label	Interpretation
< 10	Critical	Likely linkable with modest effort
10–20	Weak	Significantly narrowed candidate set
20–40	Moderate	Reasonable anonymity
40–60	Strong	Computationally difficult to link
> 60	Very Strong	Infeasible to enumerate candidates

Table 1: Privacy score interpretation guide.

A score of 40 bits implies approximately 10^{12} candidate deposit subsets, making brute-force linking computationally infeasible.

6.4 Improving Privacy Scores

Users can increase their privacy score by:

1. **Sacrificing precision:** Withdrawing slightly less than entitled increases the target range, capturing more candidate subsets.
2. **Waiting for deposits:** A larger deposit pool increases the anonymity set.
3. **Using common amounts:** Round numbers match more deposits.
4. **Batching with others:** Aggregated batches combine anonymity sets.

6.5 Limitations

The privacy score is a heuristic with known limitations:

- **Conservative estimate:** The deposit pool is a superset of unspent deposits (nullifiers cannot be linked to deposits).
- **Timing ignored:** The score does not account for temporal correlation.
- **Single withdrawal:** Scores for sequences of withdrawals may combine non-independently.

Despite these limitations, the score provides actionable guidance for privacy-conscious users.

7 Implementation and Evaluation

We implement the wormhole protocol in production on the Quantus Network, a Substrate-based blockchain [12].

7.1 Implementation Overview

Circuit Implementation: The ZK circuits are implemented in Rust using Plonky2 [11]. Key design decisions include:

- **no_std compatibility:** Core crates compile without the standard library, enabling WASM deployment.
- **Modular fragments:** Each constraint set (nullifier, storage proof, etc.) is a separate `CircuitFragment` for composability.
- **Pre-built binaries:** Circuit data is serialized with SHA-256 hash verification for deterministic deployment.

Cryptographic Primitives:

- **Hash function:** Poseidon2 [5] with 4-element (256-bit) digests over the Goldilocks field.
- **Proof system:** Plonky2 with $D = 2$ extension degree, providing approximately 100 bits of security.
- **Field:** Goldilocks prime $p = 2^{64} - 2^{32} + 1$ for efficient 64-bit arithmetic.

On-Chain Verifier: The `wormhole` pallet implements on-chain verification:

- Proof deserialization and Plonky2 verification
- Block hash validation against chain history
- Nullifier set management (storage map)
- Token minting to exit accounts

7.2 Performance Characteristics

Table 2 summarizes expected performance characteristics. Detailed benchmarks are ongoing.

Operation	**Time**	**Notes**
Leaf proof generation	∼300ms	Single withdrawal proof
Layer-0 aggregation ($N = 16$)	∼5s	Client-local, privacy-sensitive
Layer-1 aggregation ($M = 16$)	∼5s	Delegated aggregation
On-chain verification	∼50ms	Single aggregated proof
Proof size (leaf)	∼200KB	Before compression
Proof size (aggregated)	∼43KB	After recursive compression
Public inputs	21 felts	168 bytes per leaf

Table 2: Expected performance characteristics (benchmarks TBD).

Method	Per-Tx Size	5MB Block	Improvement
ML-DSA-87 signature	∼7.3KB	∼685 tx	1×
Wormhole (individual)	∼200KB	∼25 tx	0.04×
Wormhole (16×16 agg.)	∼170B/tx	∼29,400 tx	43×
Wormhole (optimistic)	∼115B/tx	∼43,750 tx	64×

Table 3: Throughput comparison for 5MB blocks.

7.3 Throughput Analysis

We compare throughput between naive post-quantum signatures and wormhole transactions:

With two-layer aggregation (256 withdrawals per on-chain proof), wormhole transactions achieve over 60× throughput improvement compared to naive ML-DSA verification.

7.4 Deployment Status

The wormhole system is deployed on the Quantus Network testnet with:

- Full circuit implementation for native token transfers
- Layer-0 and Layer-1 aggregation support
- Privacy score computation via off-chain indexer
- Wallet integration for proof generation

Multi-asset support (non-native tokens) and additional aggregation layers are planned for future releases.

8 Related Work

Privacy-Preserving Cryptocurrencies. Zcash [6] pioneered practical zero-knowledge transactions using zk-SNARKs, enabling shielded transfers that hide sender, receiver, and amount. However, Zcash relies on pairing-based cryptography vulnerable to quantum attacks and required a trusted setup ceremony. Tornado Cash [8] brought privacy to Ethereum via a mixing protocol, but similarly lacks post-quantum security. Our work extends these privacy techniques to the post-quantum setting.

Post-Quantum Blockchain Signatures. The NIST post-quantum standardization [7] selected ML-DSA (Dilithium) [4] for digital signatures. Direct adoption in blockchains faces scalability challenges due to large key and signature sizes. Hash-based signatures (SPHINCS+) offer smaller keys but even larger signatures. Our approach sidesteps these limitations by moving signature verification off-chain entirely.

ZK-Based Signature Compression. Ziggy [10] from StarkWare proposes using STARKs to verify post-quantum signatures off-chain, similar in spirit to our approach. The key difference is that Ziggy compresses *existing* signature verification, while wormhole addresses eliminate signatures entirely via the unspendable address construction. EIP-7503 [3] proposed "zero-knowledge wormholes" for Ethereum, introducing the proof-of-burn concept we build upon. Our work extends this with formal security analysis, recursive aggregation, and privacy score quantification.

Recursive SNARKs. Recursive proof composition was introduced in theoretical work and made practical by systems like Halo [11] and Plonky2. We leverage Plonky2's efficient recursion for proof aggregation. Unlike pairing-based recursive SNARKs, Plonky2's FRI-based approach [1, 2] provides post-quantum security without trusted setup.

Anonymity Set Analysis. Our privacy score draws inspiration from anonymity metrics in mix networks and differential privacy. The CLT-based subset-sum approximation relates to lattice cryptography techniques for analyzing noisy subset-sum problems. Similar heuristics appear in Tornado Cash anonymity mining and Zcash shielded pool analysis.

9 Conclusion

We introduced wormhole addresses, a novel construction enabling scalable, privacy-preserving transactions for post-quantum blockchains. By combining provably unspendable addresses with recursive zero-knowledge proofs, our protocol achieves over 60× throughput improvement compared to naive post-quantum signature verification, while providing unlinkability guarantees comparable to mixing protocols.

Our key contributions include: (1) the double-hashing wormhole address construction with formal unspendability proofs, (2) a complete ZK circuit design for privacy-preserving withdrawals, (3) a multi-layer aggregation scheme with careful privacy preservation, and (4) a production implementation on the Quantus Network using Plonky2.

The wormhole approach demonstrates that the transition to post-quantum cryptography need not sacrifice blockchain scalability. By shifting signature verification off-chain, we can simultaneously achieve post-quantum security, high throughput, and transaction privacy—properties previously considered difficult to combine.

Future Work. Several directions remain for future investigation:

- **Cross-chain bridges:** Extending wormhole proofs to verify deposits across chains.
- **Multi-asset support:** Generalizing the circuit to arbitrary token types.
- **Formal verification:** Machine-checked proofs of circuit correctness.
- **Privacy-preserving compliance:** Optional viewing keys for regulatory requirements.
- **Improved aggregation:** Deeper aggregation trees and optimized batching strategies.

As quantum computing advances, wormhole addresses offer a practical path toward quantum-resistant blockchain infrastructure without compromising on the scalability and privacy properties users expect.

References

[1] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast reed-solomon interactive oracle proofs of proximity. ICALP, 2018.

[2] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, 2018.

[3] Ethereum Community. Eip-7503: Zero-knowledge wormholes. https://eips.ethereum.org/EIPS/eip-7503, 2023.

[4] Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. TCHES, 2018.

[5] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A faster version of the poseidon hash function. Cryptology ePrint Archive, Report 2023/323, 2023.

[6] Daira Hopwood, Sean Bowe, Taylor Hornby, and Nathan Wilcox. Zcash protocol specification. https://zips.z.cash/protocol/protocol.pdf, 2023.

[7] NIST. Module-lattice-based digital signature standard (fips 204). https://csrc.nist.gov/pubs/fips/204/final, 2024.

[8] Alexey Pertsev, Roman Semenov, and Roman Storm. Tornado cash privacy solution. https://tornado.cash, 2019.

[9] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.

[10] StarkWare. Ziggy: Post-quantum signatures from starks. https://starkware.co/, 2024.

[11] Polygon Zero Team. Plonky2: Fast recursive arguments with plonk and fri. https://github.com/0xPolygonZero/plonky2, 2022.

[12] Parity Technologies. Substrate: The blockchain framework for a multichain future. https://substrate.io/, 2023.